Національний технічний університет України
«Київський політехнічний інститут імені Ігоря Сікорського»

Факультет інформатики та обчислювальної техніки
Кафедра обчислювальної техніки

ЗВІТ

з лабораторної роботи №2
з дисципліни «Технології та методика програмування вбудованих систем»

«Дослідження моделі пристрою керування двигуном постійного струму»

Варіант №4

Виконав:
студент 1 курсу, групи ІМ-51мн
Ковальов Олександр

Перевірив:
ст. викладач, к.т.н Орленко Сергій Петрович

Дата здачі: 15.03.2026

КИЇВ – 2026

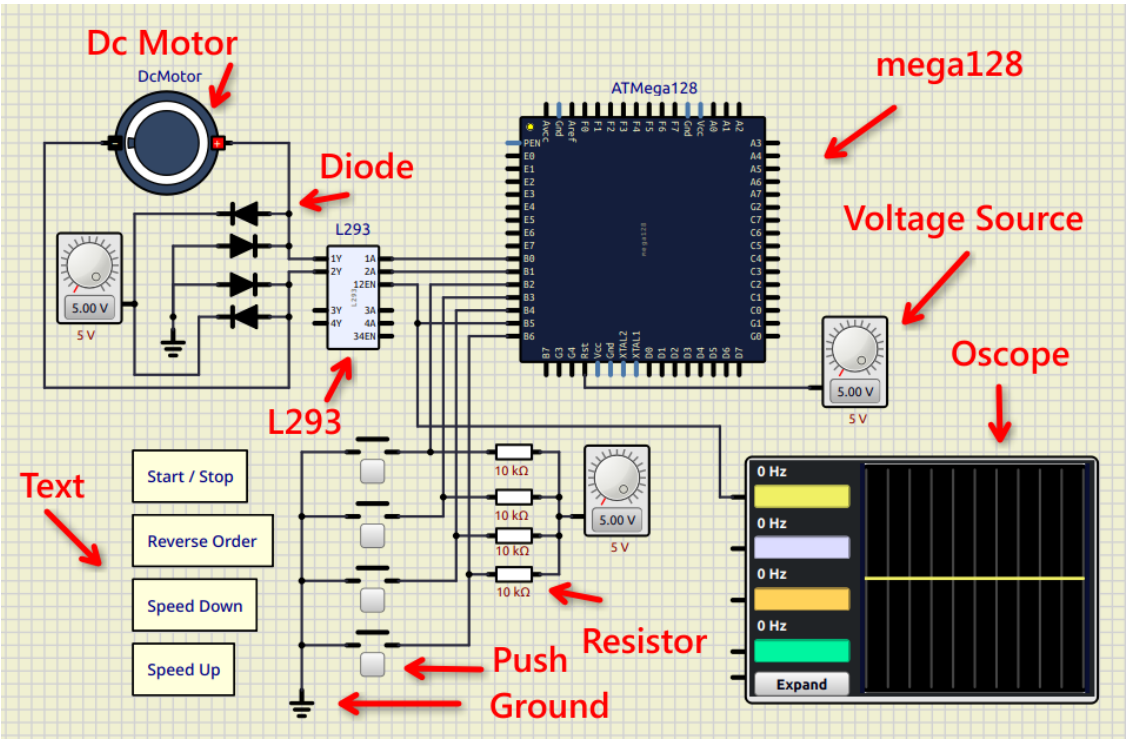
Тема. Моделювання пристрою керування двигуном постійного струму з використанням мікроконтролера сім'ї AVR.

Мета. Користуючись пакетом SimulIDE дослідити роботу пристрою керування двигуном постійного струму.

Завдання. Створити модель пристрою в пакеті SimulIDE. Розробити схему алгоритму роботи моделі та робочу програму. Створити hex-файл та підключити його до мікроконтролера. Запустити модель та виконати її дослідження згідно методичних вказівок. Зробити відповідні висновки.

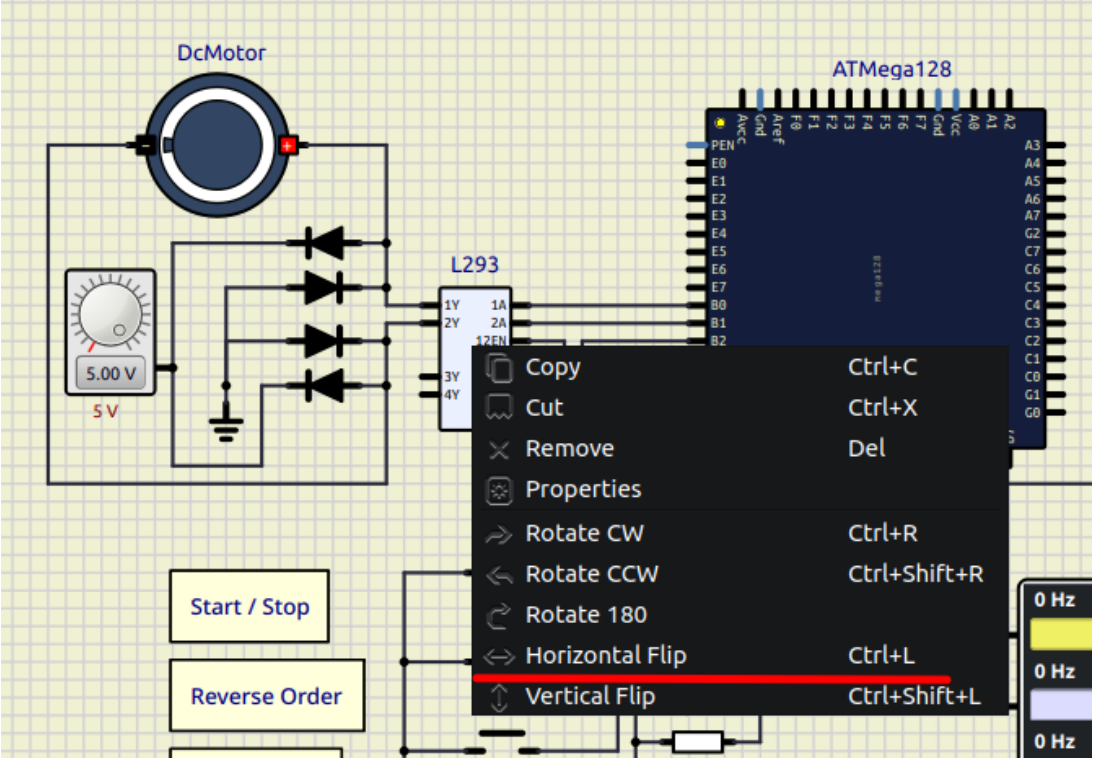
Хід роботи.

Роботу розпочато зі створення моделі пристрою керування двигуном постійного струму в середовищі SimulIDE. У якості основи було обрано мікроконтролер ATmega128. Для керування двигуном постійного струму використано драйвер L293, так як моделі L293D немає в пакеті. Оскільки драйвер не має вбудованих захисних діодів, для захисту транзисторних ключів від паразитних імпульсів було додано зовнішню обв'язку з чотирьох діодів. Двигун підключено до виходів OUT1 та OUT2 (1Y та 2Y) драйвера, а керуючі входи IN1 та IN2 (1A та 2A) з'єднано з портами PB0 та PB1 (B0 та B1) мікроконтролера. Вхід увімкнення моста EN1 (12EN) з'єднано з портом PB5, звідки подається ШІМ-сигнал. Для керування системою підключено чотири кнопки до портів PB2, PB3, PB4 та PB6 (B2, B3, B4 та B6) із використанням резисторів з опором в 10 кОм. Для візуалізації ШІМ-сигналу до лінії PB5 (B5) підключено осцилограф.

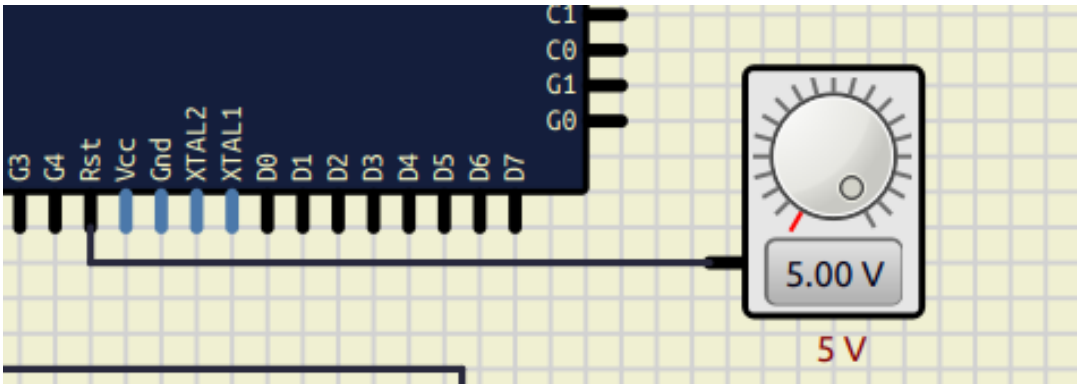


Щодо схеми діодів, то два з них підключені катодами до виходу драйверу L293, портів 1Y та 2Y, анодами до заземлення. Два інших – катодами до живлення 5 вольт, анодами до тих же портів 1Y та 2Y.

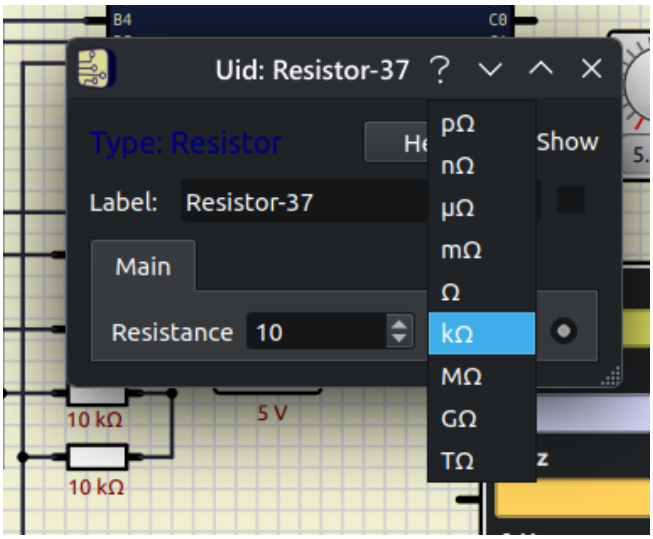
Для зручності, драйвер та два діоди були відзеркалені. Це можна зробити, якщо натиснути ПКМ на елемент, і потім обрати пункт «Horizontal Flip»:



ATmega128 має інверсний вхід скидання, який позначається як Rst. Це означає, що для нормальної роботи на ньому постійно має бути високий рівень напруги, тобто п’ять вольт. Якщо цей пін залишити непідключеним, симулятор може сприймати його стан як логічний нуль. Через це мікроконтролер постійно перебуває у стані перезавантаження, він повністю заморожений і не виконує жодного рядка коду. Звичайно, цей момент ігнорується симулятором, але під час запуску симуляції все одно відображається попередження. Тому, бажано підключити цей порт до джерела живлення, як і в першій лабораторній роботі:

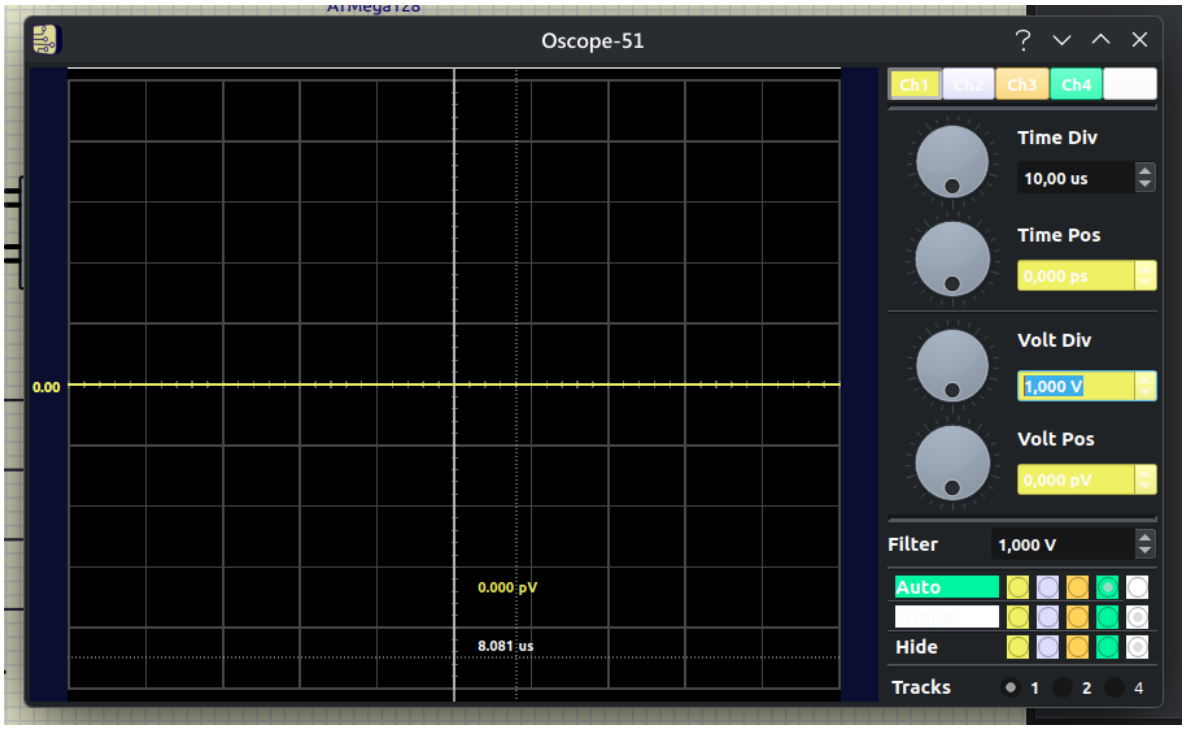


Окрім цього, для резисторів був встановлений опір 10 кОм. В меню налаштування можна зайти, якщо натиснути ПКМ → «Properties».



Цього разу осцилограф потребує лише одного треку, а саме першого, який під'єднаний до лінії між пінами B5 мікросхеми та 12EN драйверу. Встановлений час на одну поділку – 10 мікросекунд, все інше за замовчуванням (Time Pos = 0, Volt Div = 0 V., Volt Pos = 0). Звичайно, кількість треків також треба встановити – рівно один.

Якщо при симуляції перший трек відображається суцільним жовтим кольором, треба встановити час на поділку вручну, не в налаштуваннях – прокрутити колесиком миші сам осцилограф до бажаного значення, і вже потім, коли в налаштуваннях симулятор сам проставить префікс «мікро», можна встановити рівно 10.



Згідно з вхідними даними лабораторної роботи, частота нашого ШІМ-сигналу становить $f_{B5} = 25$ кГц. Це означає, що один повний період коливання триває лише $T_{B5} = 40$ мкС. В одній мілісекунді тисяча мікросекунд, тому при налаштуванні в 10 мілісекунд на одну клітинку на екрані осцилографа «втискується» аж 250 повних періодів сигналу. Вони стоять настільки щільно один до одного, що графіка просто не витримує і зливає їх в одну суцільну жовту смугу. Саме через це треба налаштувати той один єдиний трек. Якщо не налаштовувати, крім жовтої смуги можна буде побачити правильно розпізнану частоту – 25 кГц, що підтверджує правильність роботи мікроконтролера.

Наступний крок – розрахунок параметрів ШІМ-модуля згідно з вхідними даними. Частота ШІМ-сигналу задана як 25 кГц, частота тактового сигналу дорівнює 16 МГц, таймер працює в режимі Phase and Frequency Correct PWM, а початкова шпаруватість становить 2. Коефіцієнт ділення попереднього дільника обрано N = 8.

Значення модуля лічби TOP, яке визначається вмістом регістра ICR1, розраховується як відношення тактової частоти до подвоєного добутку дільника та частоти ШІМ-сигналу. Підставивши значення, отримуємо ICR1 = 40 (або 0x0028 у шістнадцятковій системі).

Період ШІМ-сигналу розраховується як обернена величина до частоти і становить 40 мікросекунд. Для забезпечення шпаруватості Q = 2 тривалість імпульсу повинна становити 20 мікросекунд. Звідси визначаємо значення регістра порівняння OCR1A, яке дорівнює 20 (або 0x0014).

Далі було розраховано значення шпаруватості при збільшенні та зменшенні значення регістра OCR1A на 5 одиниць. При OCR1A = 25 шпаруватість становить 2,667. При OCR1A = 15 шпаруватість зменшується до 1,6.

Для реалізації алгоритму керування мовою Асемблер було розроблено окремі фрагменти коду для конфігурації таймера. Програма зупинки таймера записує нульове значення у регістр TCCR1B.

```
LDI R18, $00 ; R18 = 0x00
LDI R27, $00 ; R27 = 0x00
LDI R26, $4E ; R26 = 0x4E (TCCR1B register address)
ST X, R18    ; Stop timer
```

Завантаження регістра TCCR1A вимагає встановлення бітів COM1A1 та COM1A0 в одиницю для генерації інвертованого ШІМ-сигналу.

```
LDI R17, $C0 ; R17 = 0xC0
LDI R29, $00 ; R29 = 0x00
LDI R28, $4F ; R28 = 0x4F (TCCR1A register address)
ST Y, R17    ; Set TCCR1A configurations
```

Завантаження 16-розрядного регістра ICR1 значенням 40 (0x0028) виконується шляхом послідовного запису спочатку старшого, а потім молодшого байта.

```
LDI R23, $00 ; R23 = 0x00
LDI R27, $00 ; R27 = 0x00
LDI R26, $47 ; R26 = 0x47 (ICR1H register address)
ST X, R23    ; Set high byte of ICR1
LDI R24, $28 ; R24 = 0x28
LDI R29, $00 ; R29 = 0x00
LDI R28, $46 ; R28 = 0x46 (ICR1L register address)
ST Y, R24    ; Set low byte of ICR1
```

Аналогічно виконується завантаження початкового значення 20 (0x0014) у регістр порівняння OCR1A.

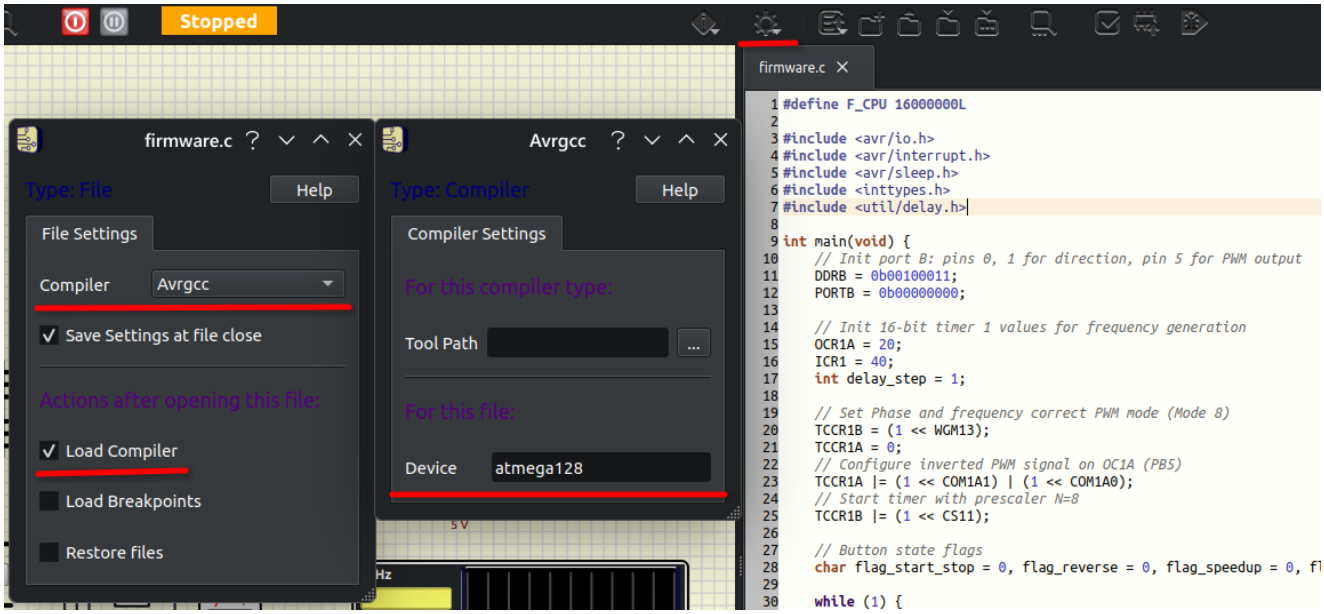
```
LDI R24, $00 ; R24 = 0x00
LDI R31, $00 ; R31 = 0x00
LDI R30, $4B ; R30 = 0x4B (OCR1AH register address)
ST Z, R24    ; Set high byte of OCR1A
LDI R25, $14 ; R25 = 0x14
LDI R31, $00 ; R31 = 0x00
LDI R30, $4A ; R30 = 0x4A (OCR1AL register address)
ST Z, R25    ; Set low byte of OCR1A
```

Запуск таймера з дільником N = 8 у режимі Phase and Frequency Correct PWM здійснюється записом значення 0x12 у регістр TCCR1B.

```
LDI R17, $12 ; R17 = 0x12
LDI R29, $00 ; R29 = 0x00
LDI R28, $4E ; R28 = 0x4E (TCCR1B register address)
ST Y, R17    ; Start timer with prescaler 8
```

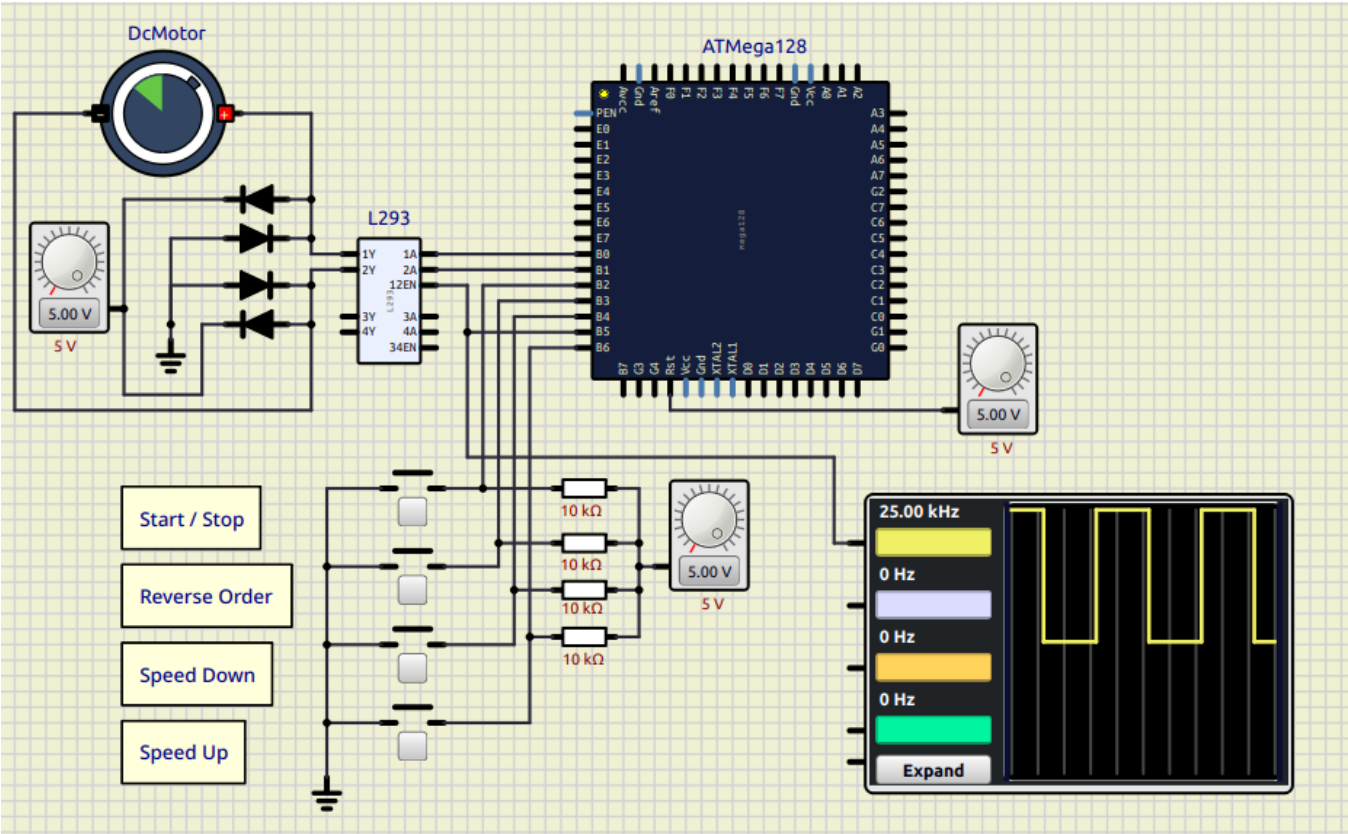
Основну логіку керування пристроєм було реалізовано мовою C. Програма ініціалізує порти введення-виведення, налаштовує апаратний таймер для генерації ШІМ-сигналу та у нескінченному циклі опитує стан кнопок. Натискання кнопки на порту PB2 викликає інверсію сигналу на порту PB0, що запускає або зупиняє двигун. Кнопка на порту PB3 змінює напрямок обертання шляхом інверсії портів PB0 та PB1. Кнопки на портах PB4 та PB6 відповідають за зміну значення регістра OCR1A, що динамічно змінює шпаруватість ШІМ-сигналу та, як наслідок, швидкість обертання двигуна.

Компіляція, як і минулого разу, була виконана засобами `avr-gcc` та SimulIDE.

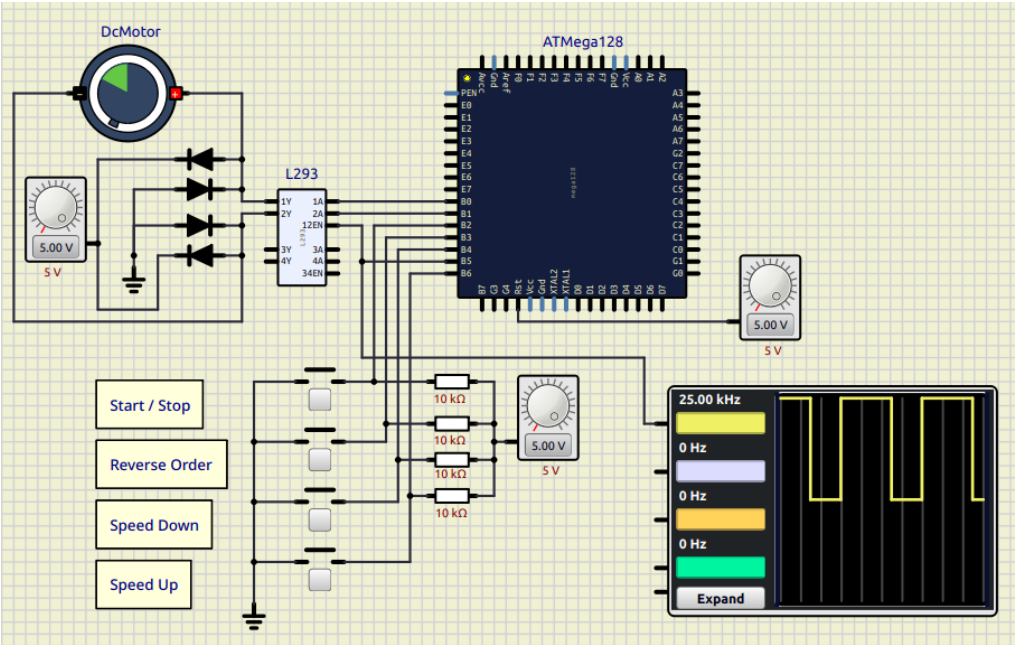


Після компіляції коду та завантаження отриманого hex-файлу у мікроконтролер було проведено тестування моделі в SimulIDE. Спостереження за осцилографом підтвердило коректну генерацію імпульсів із частотою 25 кГц. При натисканні кнопок зміни швидкості було зафіксовано відповідну зміну ширини імпульсів, що наочно демонструє процес широтно-імпульсної модуляції.

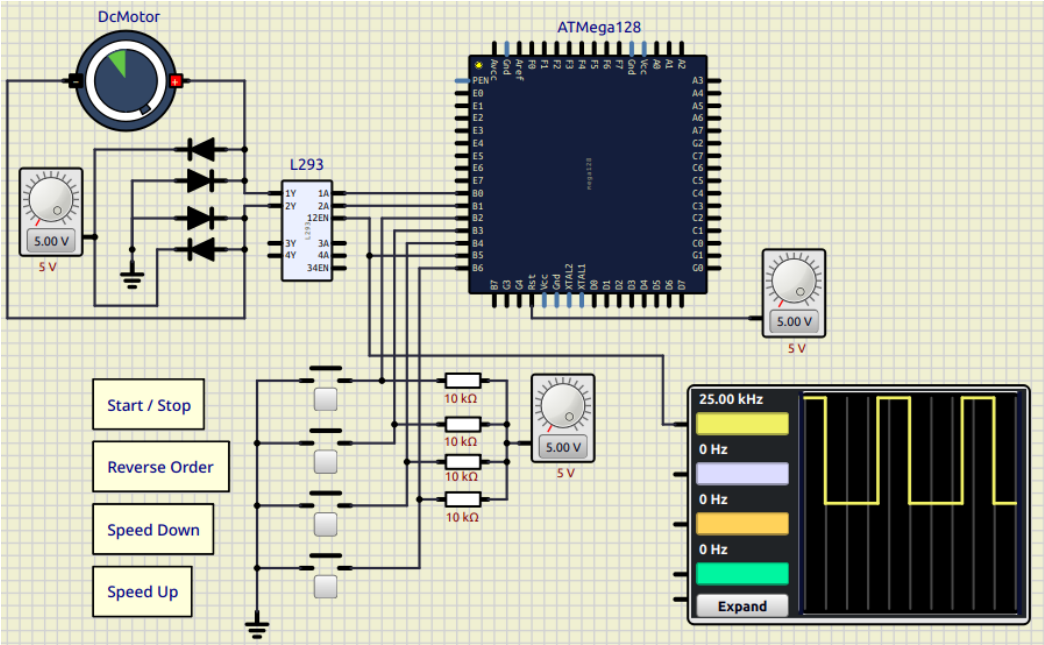
ШІМ-сигнал у початковому стані (шпаруватість $q = 2$):



ШІМ-сигнал після збільшення швидкості (шпаруватість $q = 1,6$):



ШІМ-сигнал після зменшення швидкості (шпаруватість $q = 2,67$)



Висновок. У ході виконання лабораторної роботи було успішно створено та досліджено модель пристрою керування двигуном постійного струму в середовищі SimulIDE з використанням мікроконтролера ATmega128. Було практично реалізовано керування швидкістю обертання ротора за допомогою генерації ШІМ-сигналу апаратним модулем таймера, а також забезпечено керування напрямком обертання через мікросхему мостового драйвера. Завдяки розробленій програмі мовою C перевірено вплив зміни шпаруватості ШІМ-сигналу на еквівалентну напругу та швидкість обертання двигуна. Усі теоретичні розрахунки параметрів таймера повністю підтвердилися під час симуляції.

Контрольні запитання.

1. **Для чого може використовуватись таймер/лічильник?**
   Для керування двигуном постійного струму в моделі використовується модуль таймера мікроконтролера, який працює в режимі широтно-імпульсної модуляції. Загалом він також застосовується для відліку часових інтервалів та підрахунку зовнішніх подій.
2. **Які регістри відповідають за дозвіл/заборону переривань від таймерів/лічильників?**
   За дозвіл або заборону переривань від таймерів відповідають спеціальні регістри масок переривань, такі як TIMSK (Timer/Counter Interrupt Mask Register) та ETIMSK (Extended Timer/Counter Interrupt Mask Register).
3. **Чим відрізняється режим роботи `CTC` від режиму `Normal`?**
   У режимі Normal модуль лічби дорівнює максимальному значенню $FFFF, і прапорець переповнення встановлюється при досягненні цього значення. У режимі CTC модуль лічби визначається регістрами OCRnA або ICRn, і при досягненні цього значення таймер автоматично скидається в нуль.
4. **Для рішення яких завдань краще використання режиму `Phase Correct PWM` замість `Fast PWM`?**
   Цей режим краще використовувати для керування електродвигунами, оскільки завдяки його принципу роботи кожен період ШІМ-сигналу є повністю симетричним.
5. **Для чого можуть використовуватись 16-розрядні таймери/лічильники?**
   Для формування ШІМ-сигналів в моделі використовується модуль таймера мікроконтролера. Вони дозволяють генерувати апаратні сигнали з високою точністю, вимірювати тривалість імпульсів та формувати симетричні сигнали керування.
6. **При виникненні яких подій таймери/лічильники T1, T3, T4, T5 можуть генерувати переривання?**
   Переривання можуть генеруватися при встановленні прапорця переривання OCFnx (збіг порівняння), встановленні прапорця ICFnA або OCFnA, а також при переповненні таймера, коли встановлюється прапорець TOVn.
7. **За допомогою яких регістрів виконується керування таймером/лічильником?**
   Керування виконується за допомогою регістрів TCCRnA, TCCRnB та TCCRnC, де n позначає номер відповідного таймера.

8. **Які сигнали можуть використовуватись в якості тактового сигналу `fclkn` для таймерів/лічильників T1, T3, T4 і T5?**

   В якості тактового сигналу може використовуватися внутрішня частота підсистеми введення/виведення безпосередньо або через попередній дільник на 8, 64, 256 чи 1024, а також зовнішній сигнал на виводі Tn з лічбою за наростаючим або спадаючим фронтом.

9. **Яким чином здійснюється вибір джерела тактового сигналу, а також запуск і зупинка таймерів/лічильників?**

   Вибір джерела тактового сигналу та запуск здійснюється комбінацією бітів CSn2, CSn1 та CSn0 у регістрі TCCRnB. Якщо всі ці три біти скинуті в нуль, таймер зупиняється.

10. **Як увімкнути режим роботи `Normal` таймера/лічильника T3? Для чого може використовуватись даний режим роботи?**

    Для увімкнення режиму Normal потрібно встановити в нуль біти WGM33, WGM32, WGM31 та WGM30. Цей режим використовується для класичного відліку часу до переповнення без впливу на виходи ШІМ.

11. **Що таке роздільна здатність лічильника таймера?**

    Роздільна здатність визначається кількістю бітів, які використовуються для підрахунку. Для 16-розрядного таймера це означає можливість рахувати від 0 до 65535, що забезпечує значно вищу плавність зміни шпаруватості ШІМ-сигналу порівняно з 8-розрядними таймерами.

12. **У чому полягає різниця між роботою 8-розрядних та 16-розрядних таймерів/лічильників у режимі `Fast PWM`?**

    16-розрядні таймери дозволяють встановлювати змінний модуль лічби TOP за допомогою регістрів ICRn або OCRnA, тоді як базові 8-розрядні таймери зазвичай працюють з фіксованим TOP.

13. **Завдяки чому в режимі `Phase and Frequency Correct PWM` кожен період сигналу є повністю симетричним?**

    Симетричність досягається завдяки тому, що таймер рахує за подвійним схилом: спочатку лічба йде вгору до досягнення значення регістра ICRn, а потім відразу вниз до нуля.

14. **Поясніть, що таке ШІМ-сигнал?**

    Це сигнал з широтно-імпульсною модуляцією, який дозволяє регулювати середню величину потужності, що надходить на двигун.

15. **Як обчислюється шпаруватість ШІМ-сигналу?**

    Шпаруватість обчислюється як відношення періоду ШІМ-сигналу до три-

валості самого імпульсу.

16. **Як впливає значення шпаруватості на швидкість обертання двигуна?**
Чим менше шпаруватість, а відповідно більше величина відношення тривалості імпульсу до періоду, тим більша напруга надходить на двигун, який буде обертатись із більшою швидкістю.

17. **Наведіть та поясніть модель пристрою керування двигуном.**
Модель складається з мікроконтролера ATmega128, драйвера L293, електродвигуна постійного струму та чотирьох кнопок керування, до яких підведена напруга живлення через підтягуючі резистори. Мікроконтролер генерує ШІМ-сигнал та логічні рівні керування напрямком, які посилюються драйвером і подаються на двигун.

18. **Наведіть та поясніть схему алгоритму роботи моделі.**
Алгоритм починається з налаштування портів виведення та ініціалізації таймера для генерації ШІМ-сигналу. Далі програма входить у нескінченний цикл, де постійно опитує стан кнопок: залежно від активної кнопки інвертуються керуючі лінії напрямку або змінюється значення регістра OCR1A для регулювання шпаруватості.

19. **Наведіть та поясніть робочу програму на мові Асемблер, що керує моделлю.**
Програма на мові Асемблер реалізує низькорівневе завантаження значень у регістри керування. Спочатку зупиняється таймер записом нуля у TCCR1B, потім налаштовується регістр TCCR1A на генерацію інвертованого ШІМ-сигналу, після чого завантажуються 16-бітні значення у регістри ICR1 та OCR1A, і врешті таймер запускається записом конфігурації дільника у TCCR1B.

20. **Наведіть та поясніть робочу програму на мові СІ, що керує моделлю.**
Програма на мові С виконує ініціалізацію порту B шляхом запису конфігурації в регістр DDRB, налаштовує таймер на режим Phase and Frequency Correct PWM, та за допомогою умовних конструкцій if обробляє натискання кнопок, змінюючи стани виходів PORTB та значення регістра OCR1A із затримкою у 10 мілісекунд для усунення брязкоту контактів.

Лістинг.

firmware.c

```c
#define F_CPU 16000000L

#include <avr/io.h>
#include <avr/interrupt.h>
#include <avr/sleep.h>
#include <inttypes.h>
#include <util/delay.h>

int main(void) {
    // Init port B: pins 0, 1 for direction, pin 5 for PWM output
    DDRB = 0b00100011;
    PORTB = 0b00000000;

    // Init 16-bit timer 1 values for frequency generation
    OCR1A = 20;
    ICR1 = 40;
    int delay_step = 1;

    // Set Phase and frequency correct PWM mode (Mode 8)
    TCCR1B = (1 << WGM13);
    TCCR1A = 0;
    // Configure inverted PWM signal on OC1A (PB5)
    TCCR1A |= (1 << COM1A1) | (1 << COM1A0);
    // Start timer with prescaler N=8
    TCCR1B |= (1 << CS11);

    // Button state flags
    char flag_start_stop = 0, flag_reverse = 0, flag_speedup = 0, flag_speeddown = 0;

    while (1) {
        // Start-Stop button action handling (connected to PB2)
        if (!(PINB & (1 << PB2))) {
            flag_start_stop = 1;
            _delay_ms(10);
        }
        if ((flag_start_stop == 1) && (PINB & (1 << PB2))) {
            PORTB ^= (1 << PB0);
            flag_start_stop = 0;
        }

        // Reverse direction button action handling (connected to PB3)
        if (!(PINB & (1 << PB3))) {
            flag_reverse = 1;
            _delay_ms(10);
        }
        if ((flag_reverse == 1) && (PINB & (1 << PB3))) {
            PORTB ^= (1 << PB0) | (1 << PB1);
            flag_reverse = 0;
        }

        // Speed down button action handling (connected to PB4)
        if (!(PINB & (1 << PB4))) {
            flag_speeddown = 1;
            _delay_ms(10);
        }
        if ((flag_speeddown == 1) && (PINB & (1 << PB4))) {
            if (OCR1A != 40) {
                OCR1A += delay_step;
```

```
            }
            flag_speeddown = 0;
        }

        // Speed up button action handling (connected to PB6)
        if (!(PINB & (1 << PB6))) {
            flag_speedup = 1;
            _delay_ms(10);
        }
        if ((flag_speedup == 1) && (PINB & (1 << PB6))) {
            if (OCR1A != 0) {
                OCR1A -= delay_step;
            }
            flag_speedup = 0;
        }
    }
}
```